The betaMixPy package

Haim Bar

November 8, 2022

1 Introduction

The betaMixPy package is used to find edges in gene networks using co-expression data. The betaMix method is built on the insightful results of [1] who showed that two random vectors are approximately perpendicular with high probability, if the dimension of the space is sufficiently large. The pair-wise correlations between pairs of predictors are equal to the cosine of the angles between the pairs. From these angles we compute $z_j = \sin^2(\theta)$ and fit a mixture of two beta distributions. For pairs of random vectors (the null set) the distribution of z_j is Beta((N-1)/2, 1/2), where N is the sample size. The nonnull set is assumed to follow a Beta(a,b) distribution, and using the EM algorithm we estimate a,b, and the proportion, p_0, of the null set of pairs. The betaMix function determines a threshold which will control the error rate given by the user. Any z_j below that threshold corresponds to a significantly correlated pair of predictors (an edge in the graphical model.) If the N samples can be assumed to be independent, set the parameter `ind` to True. If it is set to False, the null set follows a Beta((ν-1)/2, 1/2) distribution and ν (the effective sample size) is estimated from the data.

The input to the program is a normalized expression matrix, with predictors/genes (which are the nodes in the graph) in the rows, and samples in the colums (this is the input that numpy's corrcoef expects).

With a large number of predictors, P, the estimation may be slow, so it is recommended to set the parameter `subsamplesize` to something smaller than $P(P-1)/2$. The minimum allowed by the program is 20,000. Using anything smaller will cause betaMix to fit the model to all $P(P-1)/2$ pairs.

The betaMix package requires the following Python packages: statistics, matplotlib, pandas, networkx, numpy, scipy.

2 Simulated Data

The data folder in this repository contains four simulated datasets as csv files:

1. `cls_0200_1000_03_0050.csv` contains 1000 predictors, and $N = 200$ samples. The predictors come in 20 clusters, with 50 predictors in each cluster, such that the correlation of pairs within cluster is 0.3, and zero across clusters.

2. `rbk_0500_0500_03.csv` Similar to the previous item, except that the clusters have random sizes, and $P = 500$, and $N = 500$.

3. `hub_0500_0500_03_0025.csv` contains 500 predictors, and $N = 500$ samples. The predictors are arranged as 25 hubs, so that each hub consists of 20 predictors of which one is the central node and all other nodes have a 0.3 correlation with it. The correlation of pairs across hubs is zero.

4. `hub_0200_1000_09_0020.csv` Similar to the previous item, except that $P = 1000$, $N = 200$ and the correlation between a central node and all other nodes in a hub is 0.9.

We demonstrate the usage of the package with the first configuration (`i=0` in the code below).

```
import pandas as pd
import betaMixPy as bx
import matplotlib.pyplot as plt
```

```
prefix = "../data/"
files = ["cls_0200_1000_03_0050", "hub_0500_0500_03_0025", "hub_0200_1000_09_0020",
            "rbk_0500_0500_03"]
i = 0
M = pd.read_table(prefix+files[i]+".csv", sep=",", index_col=0)
M = M.transpose()
N = M.shape[0]
sineAngleMat, z_j, m0, p0, ahat, bhat, etahat, ppthr, P, edges, bmax = bx.betaMix(M,
        subsamplesize=50000, ind=True, ppr=0.01)
```

Note that we transpose the data matrix, since the betaMix method treats the data as P vectors in an R^N. We set the null posterior probability threshold to `ppr=0.01`, and we set `subsamplesize=50000` which means that in the fitting process we take a random sample of size 50000 pairwise correlations, rather than using all $P(P-1)/2$ possible pairs. We also set `ind=True` so that the N samples are assumed to be independent when we fit the model (in this case, the null distribution is determined exactly by the sample size).

After fitting the model, we can obtain a summary of the parameter estimates and a plot of the fitted mixture distribution on top of the actual data histogram. This is done with the following code:

```
bx.shortSummary(P, N, bmax, ahat, bhat, etahat, ppthr, edges, p0)
bx.plotFittedBetaMix(z_j, p0, ahat, bhat, etahat, ppthr, bmax, file="../figures/fitted.pdf")
```

The model-fit summary in this case is summarized as follows:

```
Fitting the model...

No. Nodes: 1000
Possible no. edges: 499500
No. edges detected = 15742
Detection threshold: 0.92
Pr(null): 0.9588
Nonnull support: [0, 0.95 )
Nonnull parameters (a, b, eta): ( 65.28 ,  4.39 ,  99.5 )
```

The fitted distribution plot is shown on the log-scale (Figure 1), where the null distribution is linear (depicted as a light blue line.) The nonnull component (shown in purple) is obtained from the estimates `ahat`, `bhat`, and `bmax` (the upper bound of the support of non-null component). The red dotted line shows the cutoff, such that all the pairs which yield a z_j statistic less than this threshold are considered nonnull. In this case, all the nonnull pairs were generated with the same correlation so the beta distribution fits the nonnull component very well but, in general it doesn't have to be this way – the important thing is that for true nonnull pairs, the nonnull component will be much more likely than the null distribution.

In order to cluster the predictors we first have to generate the adjacency matrix and find graph components by running the following code

```
adjM = bx.getAdjMat(sineAngleMat, ppthr)
df1 = bx.graphComponents(adjM, minCtr=2, labels=M.index)
```

Then, we can show the pairwise correlations as a bitmap, as in Figure 2.

```
bx.plotBitmapCC(adjM, ppthr, file="../figures/CorMatEst.pdf")
```

We can clearly see in Figure 2 that the algorithm detects the correct correlation structure (the yellow squares along the diagonal). The true correlation is modest ($\rho = 0.3$) and $P = 1000$ is quite large, and greater than $N = 200$, but still, most of the true correlations are detected. In similar simulations, where either ρ is greater, or N is larger the true positive rate approaches 1.

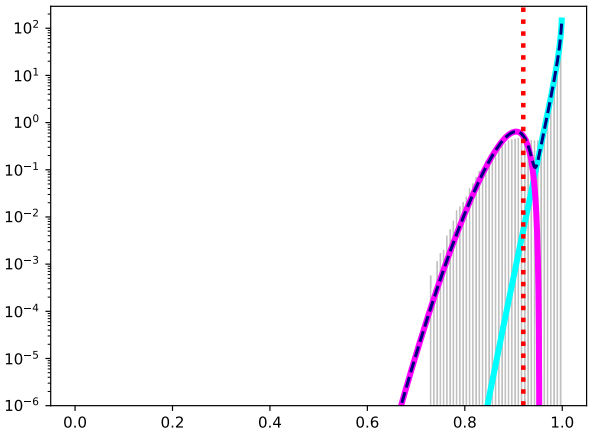


Figure 1: Simulation 1 – $N = 200$, $P = 1000$, 20 clusters with 50 predictors in each cluster, such that the true correlation of pairs within cluster is 0.3, and zero across clusters.

Furthermore, there are no yellow dots outside the diagonal of blocks, since our method yields an excellent false detection rate.

To visualize a certain cluster, we use the following code, which yields Figure 3.

```
bx.plotCluster(adjM, 1, df1, labels=True, nodecol="#DDFF00F0", file="../figures/Cluster.pdf")
```

3 Case Study – Tomato Seeds

We use the tomato metabolomic data from www.metabolomicsworkbench.org website, Study ID ST000561, introduced in [2]. There are two groups – 'dry seeds' and 'six-hour imbibed seeds'. Each group has $N = 50$, and a total of $P = 68$ metabolites. We construct the metabolite network for

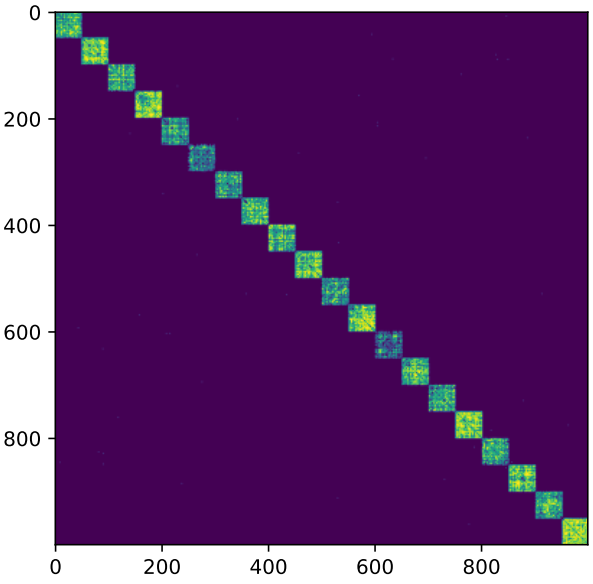


Figure 2: Simulation 1 – the inferred adjacency matrix after fitting the betaMix model.

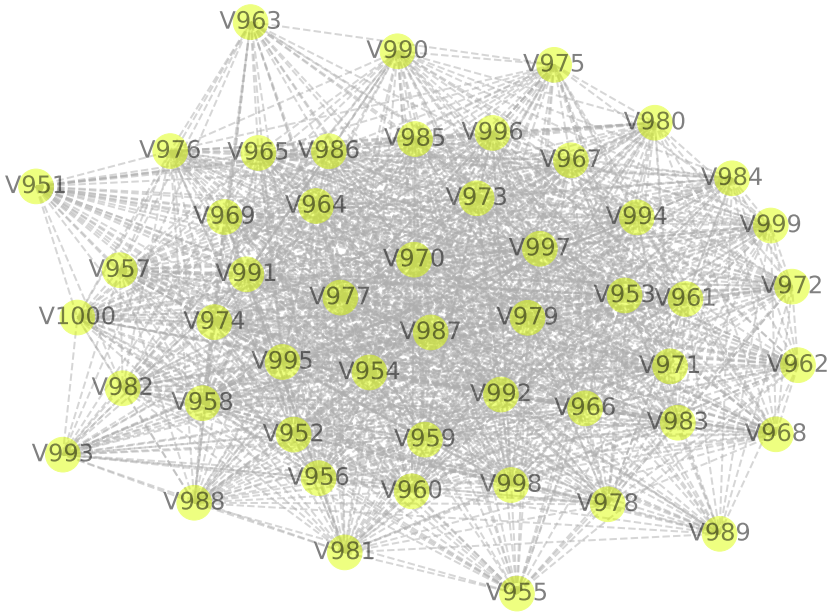
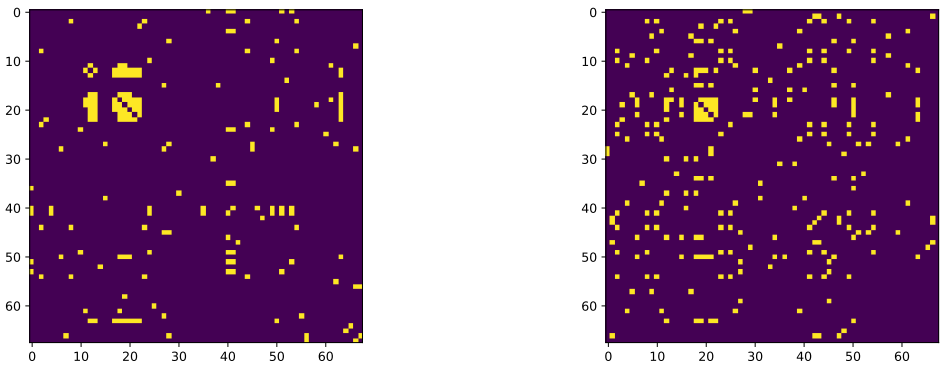


Figure 3: Simulation 1 – cluster 1.

each group separately. The code for the control (dry seeds) group is given below. The code for the second group is similar (and available in the GitHub repository.)

```
import pandas as pd
import betaMixPy as bx
import matplotlib.pyplot as plt
M0 = pd.read_table("../data/DrySeeds.csv", sep=",", index_col=0)
M0 = M0.transpose()
N0 = M0.shape[0]
sineAngleMat, z_j, m0, p0, ahat, bhat, etahat, ppthr, P, edges, bmax = bx.betaMix(M0,
        subsamplesize=50000, ind=True, ppr=0.01)
adjM0 = bx.getAdjMat(sineAngleMat, ppthr)
df0 = bx.graphComponents(adjM0, minCtr=2, labels=M0.index)
```

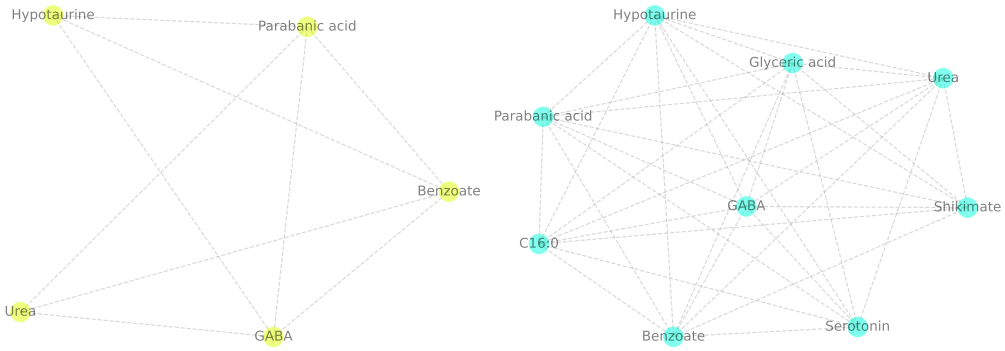
Figure 4 shows the bitmap representation of the two networks. The metabolites in the Dry Seed group are much less connected than in the Six-hour Imbibed group.



(a) Dry Seeds

(b) Six-hour Imbibed

Figure 4: Bitmap representation of the connectivity among metabolites.



(a) Dry Seeds

(b) Six-hour Imbibed

Figure 5: The network component containing the GABA metabolite in each group.

Figure 5 shows the cluster which contains the GABA metabolite in both groups. Both clusters are highly connected, but in the Dry Seed group (left) the cluster consists of five metabolites, whereas in the Six-hour Imbibed group four additional metabolites are found to be in the GABA cluster. All the edges in Figure 5(a) are also detected to be present in Figure 5(b).

References

[1] Peter Frankl and Hiroshi Maehara. Some geometric applications of the beta distribution. *Annals of the Institute of Statistical Mathematics*, 42:463–474, 1990.

[2] Rashid Kazmi, Leo Willems, Ronny Joosen, Noorullah Khan, Wilco Ligterink, and Henk Hilhorst. Metabolomic analysis of tomato seed germination. *Metabolomics*, 13, 12 2017. doi10.1007/s11306-017-1284-x.